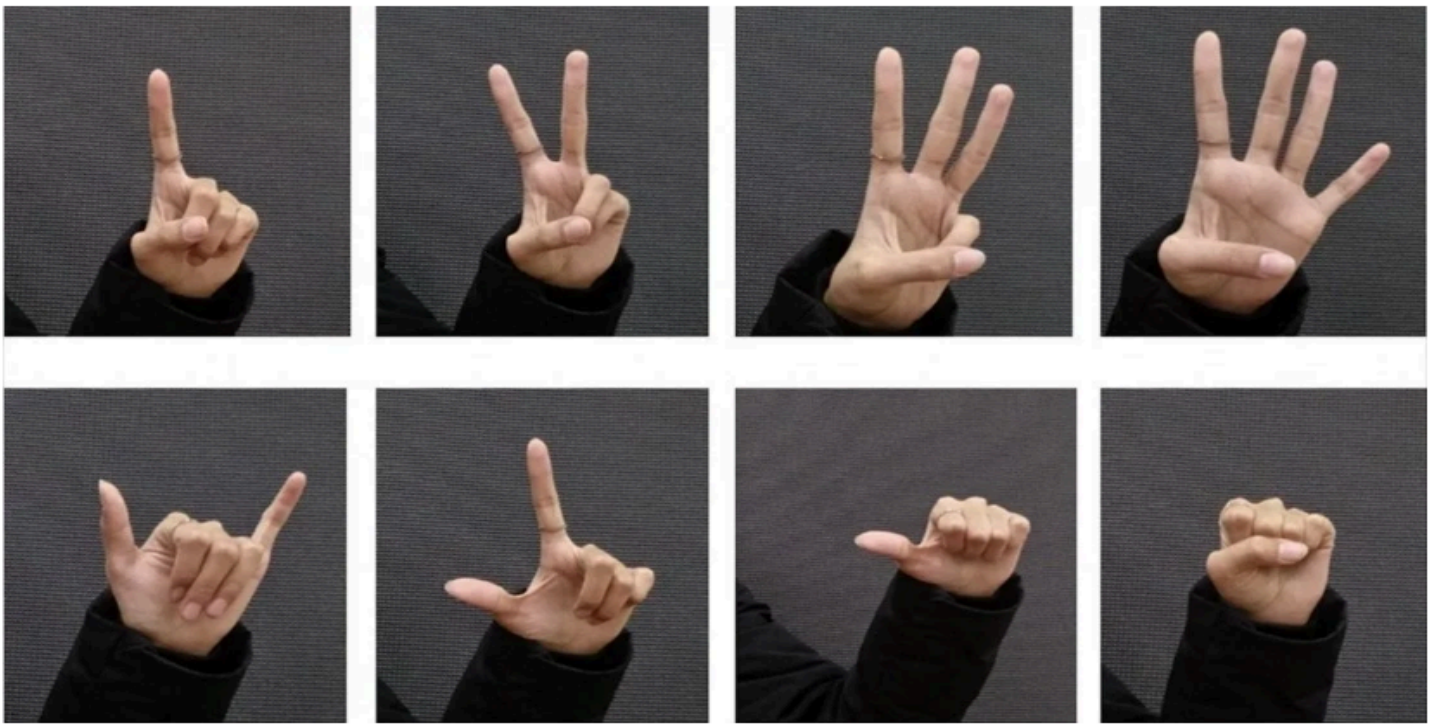
HAND-GESTURE RECOGNITION

MICRO-PROJECT



Hand Gesture Recognition Using MATLAB

To implement hand gesture recognition using CNN trained on gesture images.

Database: The first step in designing the system is to collect a dataset of hand gestures.

Dataset contains 50 images (each of five gestures) is organized into separate files to train and validate.

- **fist**
- **thumbs-up**
- **nope**
- **no-sign**
- **hiee**

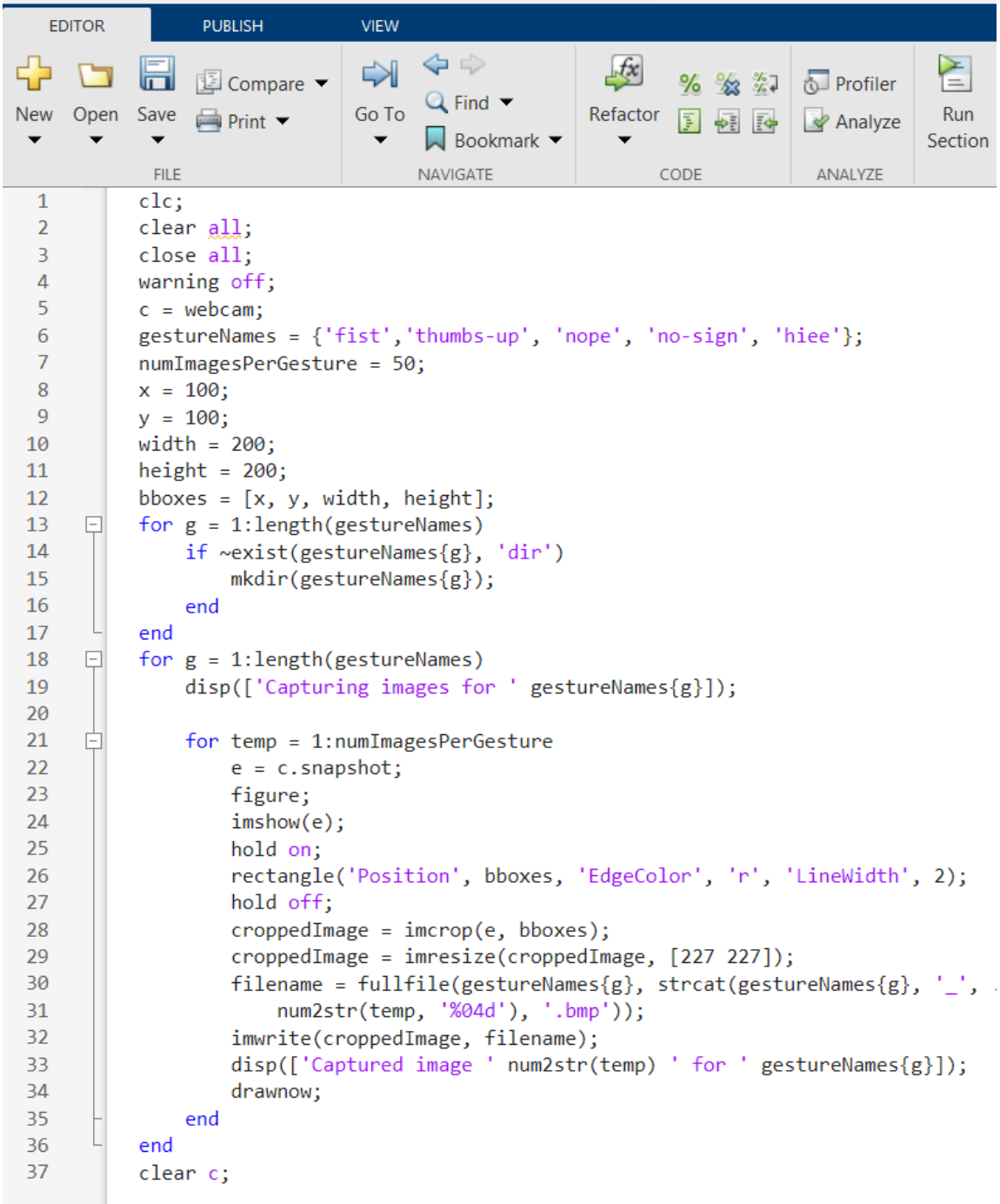
Software Used:

- **MATLAB**: The core software for implementing, training, and evaluating the CNN model.
- **MATLAB Deep Learning Toolbox**: Provides functions for building and training CNNs.
- **Webcam**: Used to capture live video for real-time gesture recognition.

Steps in Code:

1. **Input data**: Collect and organize images of each hand gesture into separate files.
2. **Preprocess**: Resize images to match the input size of the neural network
3. **Splitting of dataset**: Split the dataset into training (80%) and validation (20%) sets.
4. **Data Augmentation**: Augment the dataset to increase model accuracy.
5. **Model Training**: Train the custom Convolutional Neural Network (CNN)
6. **Model Evaluation**: Validate the trained model on the validation set to check performance.
7. **Testing**: Use real-time data to test the trained model and display the predicted gesture.
8. **Development**: Develop the system for real-time hand gesture recognition using a webcam.

MATLAB Code: (Dataset entry)



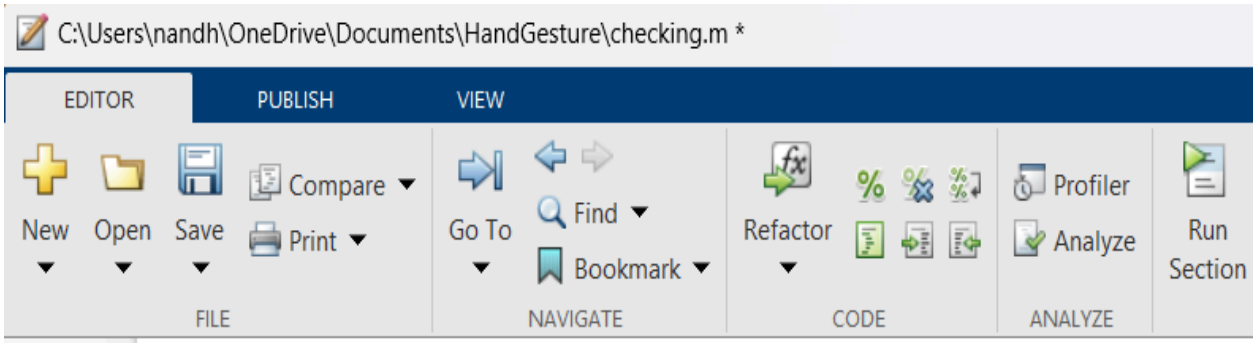

```
clc;
clear all;
close all;
warning off;
c = webcam;
gestureNames = {'fist','thumbs-up', 'nope', 'no-sign', 'hiee'};
numImagesPerGesture = 50;
x = 100;
y = 100;
width = 200;
height = 200;
bboxes = [x, y, width, height];
for g = 1:length(gestureNames)
    if ~exist(gestureNames{g}, 'dir')
        mkdir(gestureNames{g});
    end
end
for g = 1:length(gestureNames)
    disp(['Capturing images for ' gestureNames{g}]);

    for temp = 1:numImagesPerGesture
        e = c.snapshot;
        figure;
        imshow(e);
        hold on;
        rectangle('Position', bboxes, 'EdgeColor', 'r', 'LineWidth', 2);
        hold off;
        croppedImage = imcrop(e, bboxes);
        croppedImage = imresize(croppedImage, [227 227]);
        filename = fullfile(gestureNames{g}, strcat(gestureNames{g}, '_',
            num2str(temp, '%04d'), '.bmp'));
        imwrite(croppedImage, filename);
        disp(['Captured image ' num2str(temp) ' for ' gestureNames{g}]);
        drawnow;
    end
end
clear c;
```

This code is used to capture & save images of different hand gestures into separate folders using a webcam. Each gesture corresponds to a directory where images are being stored for further training and validation

Code for Data Preparation, Preprocessing & Dataset Splitting

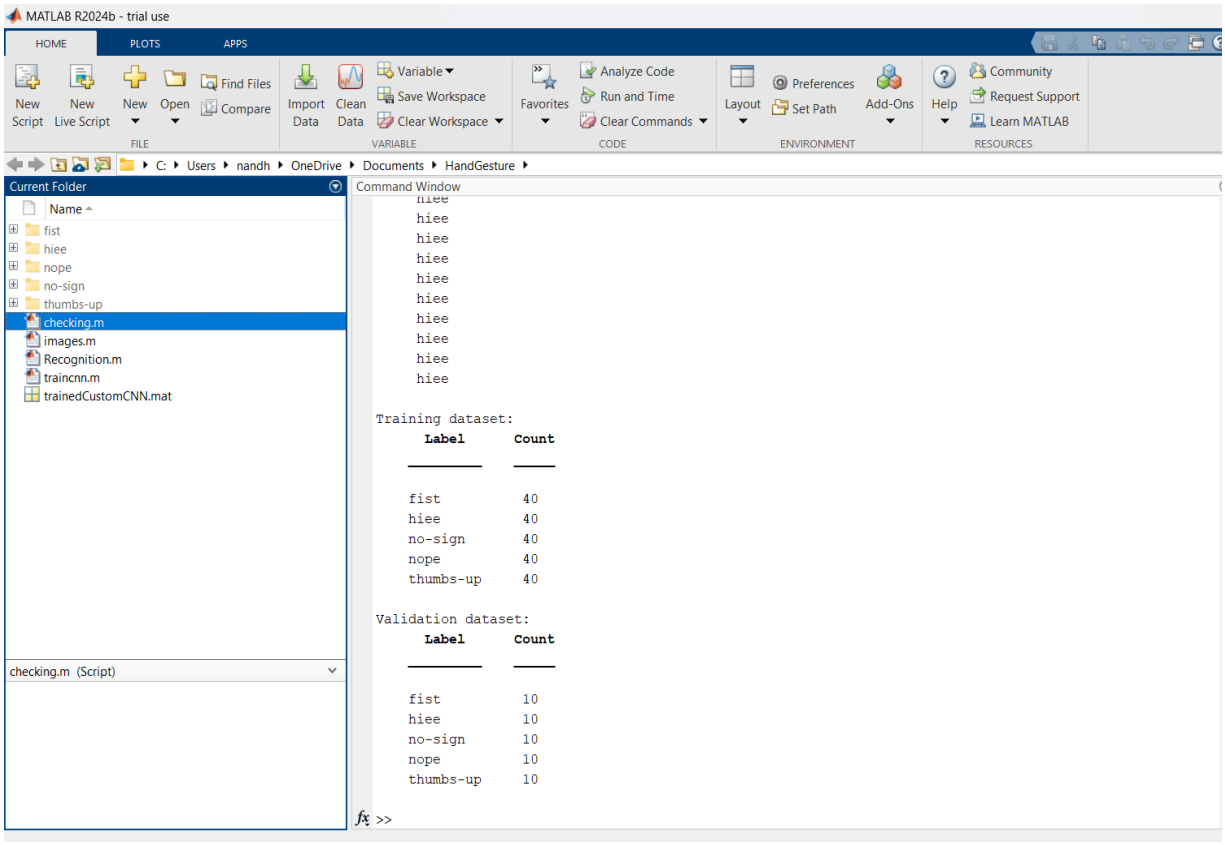
This MATLAB code is designed to prepare, process and split the dataset images for training and validation.

1. **Data Preparation**: When the images are organized into folder of specific hand gestures, the "`imageDatastore`" is used to load the mages and assign labels based on the name of the folder given.
2. **Preprocessing**: Each image in the folder is resized to size of 227x227 pixels to make the task much easier to for implementation.
3. **Dataset Splitting**: The data is split into training (80%) and validation (20%) sets. The images are shuffled to ensure randomness, so that the dataset is balanced and split properly hsb jbygknnohello



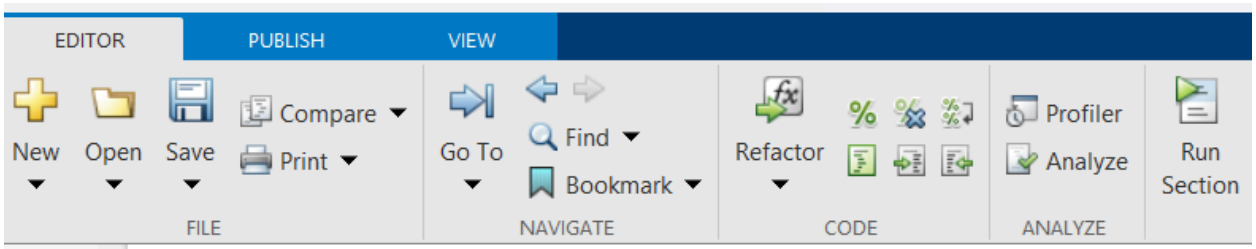

```
1   clc;
2   clear all;
3   close all;
4   gestureNames = {'fist','thumbs-up', 'nope', 'no-sign', 'hiee'};
5   numGestures = length(gestureNames);
6   imd = imageDatastore(gestureNames, 'LabelSource', 'foldernames',
7       'IncludeSubfolders', true);
8   imd.ReadFcn = @(filename)imresize(imread(filename), [227 227]);
9   [imdTrain, imdValidation] = splitEachLabel(imd, 0.8, 'randomized');
10  disp(imdTrain.Labels);
11  disp('Training dataset:');
12  disp(countEachLabel(imdTrain));
13  disp('Validation dataset:');
14  disp(countEachLabel(imdValidation));
15
```

OUTPUT:

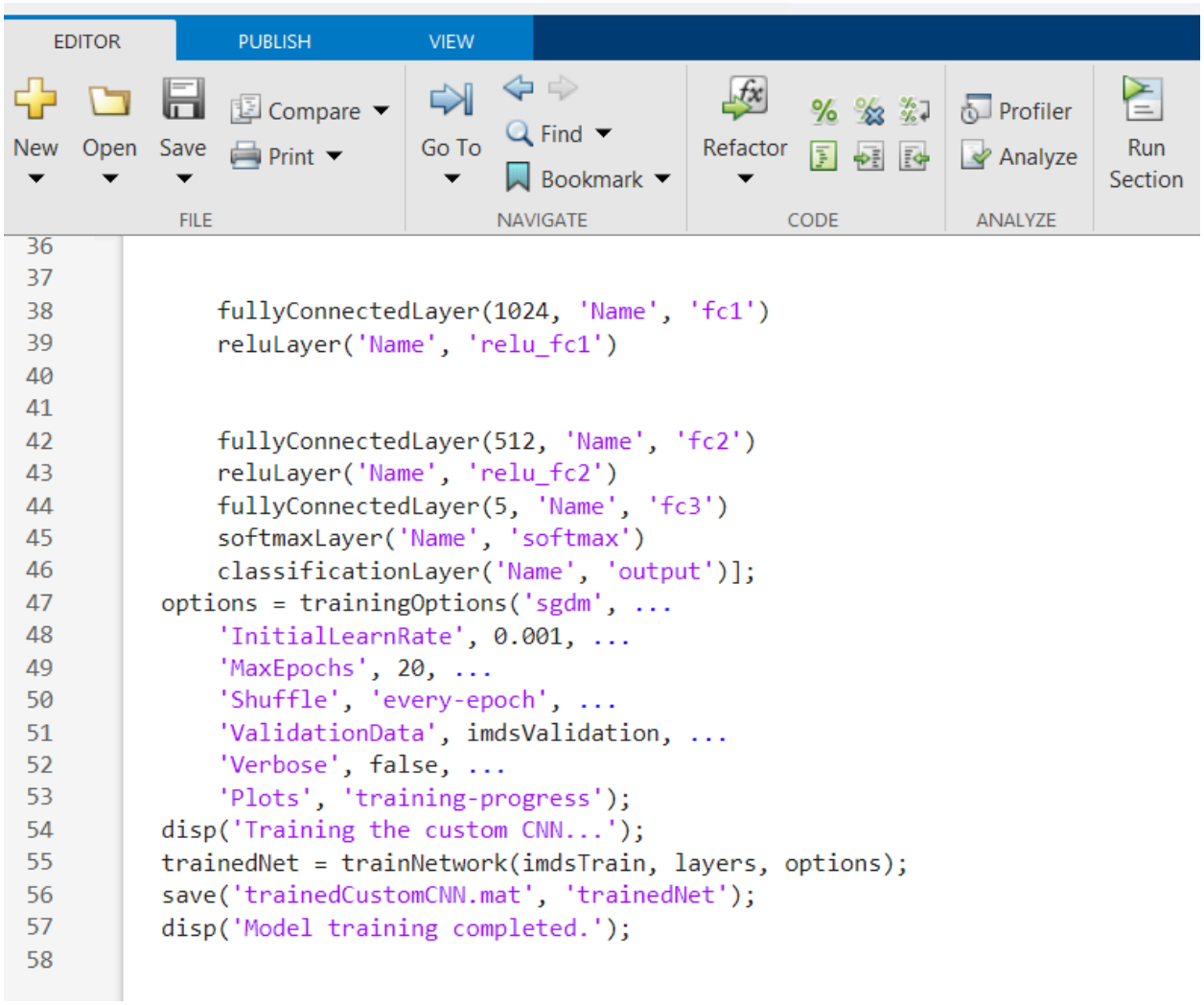


Code for training a CNN (Convolutional Neural Network) model:

This MATLAB code is used to model Convolutional Neural Network (CNN);
It will classify hand gestures based on the image dataset that we have prepared earlier. It uses a CNN architecture with five convolutional blocks (convolutional layer applies filters to input, batch normalization layer that normalizes activations to speed up training and improve stability. A ReLU activation function that introduces non linearity) and adds several fully connected layers to optimise the proper results. The network is trained on the pre-processed gesture images and then saved for future use.

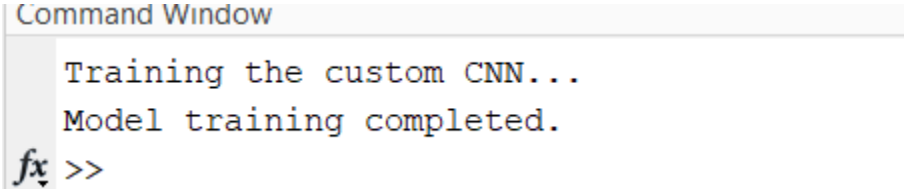


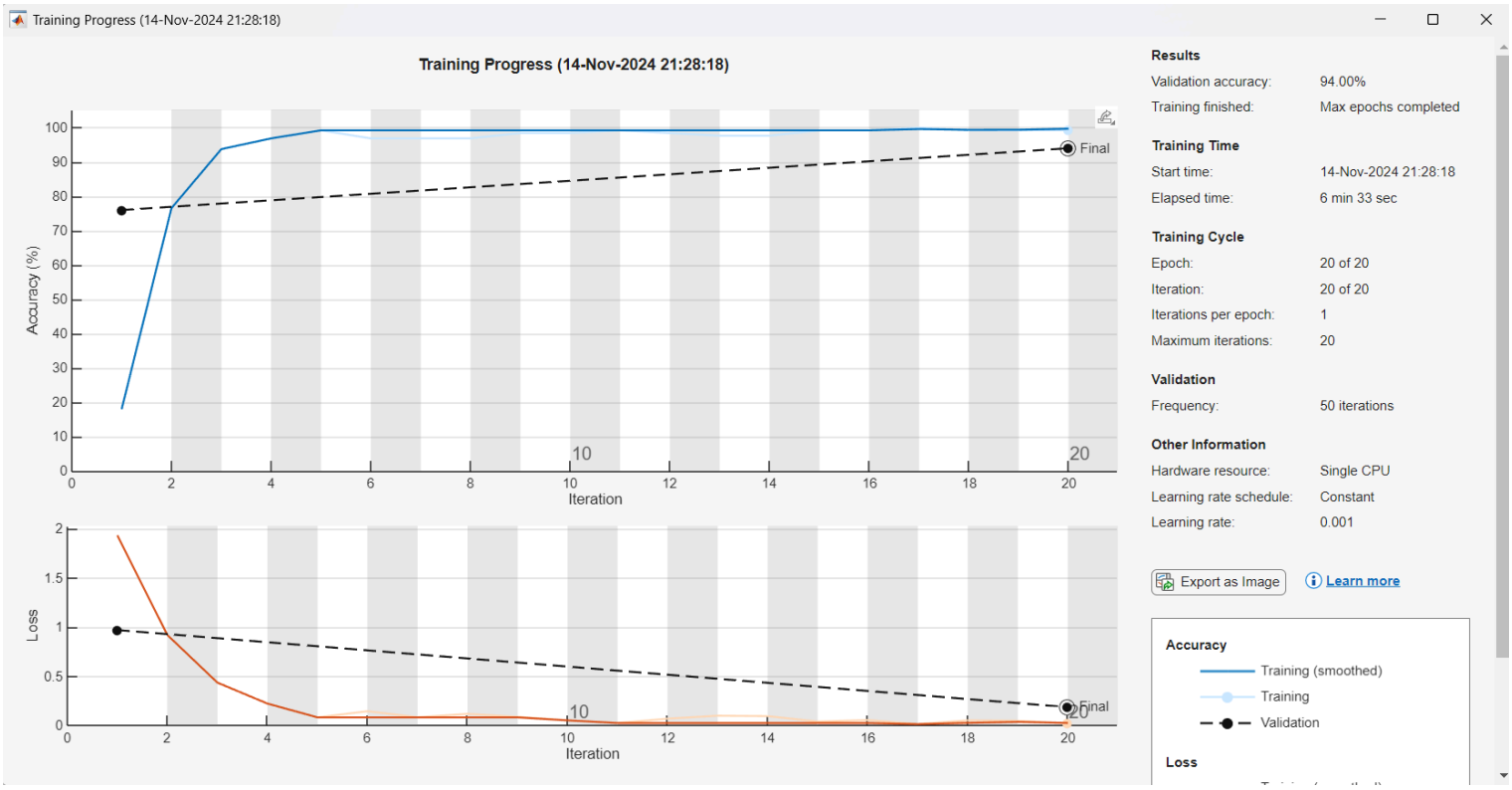

```
clc;
clear all;
close all;
gestureNames = {'fist','thumbs-up', 'nope', 'no-sign', 'hiee'};
imds = imageDatastore(gestureNames, 'LabelSource', 'foldernames',
    'IncludeSubfolders', true);
imds.ReadFcn = @(filename)imresize(imread(filename), [227 227]);
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.8, 'randomized');
layers=[imageInputLayer([227 227 3],'Name','input','Normalization','none')
    convolution2dLayer(3, 32, 'Padding', 'same', 'Name', 'conv1')
    batchNormalizationLayer('Name', 'BN1')
    reluLayer('Name', 'relu1')
    maxPooling2dLayer(2, 'Stride', 2, 'Name', 'maxpool1')


    convolution2dLayer(3, 64, 'Padding', 'same', 'Name', 'conv2')
    batchNormalizationLayer('Name', 'BN2')
    reluLayer('Name', 'relu2')
    maxPooling2dLayer(2, 'Stride', 2, 'Name', 'maxpool2')

    convolution2dLayer(3, 128, 'Padding', 'same', 'Name', 'conv3')
    batchNormalizationLayer('Name', 'BN3')
    reluLayer('Name', 'relu3')
    maxPooling2dLayer(2, 'Stride', 2, 'Name', 'maxpool3')

    convolution2dLayer(3, 256, 'Padding', 'same', 'Name', 'conv4')
    batchNormalizationLayer('Name', 'BN4')
    reluLayer('Name', 'relu4')
    maxPooling2dLayer(2, 'Stride', 2, 'Name', 'maxpool4')


    convolution2dLayer(3, 512, 'Padding', 'same', 'Name', 'conv5')
    batchNormalizationLayer('Name', 'BN5')
    reluLayer('Name', 'relu5')
    maxPooling2dLayer(2, 'Stride', 2, 'Name', 'maxpool5')

```




```
36
37
38          fullyConnectedLayer(1024, 'Name', 'fc1')
39          reluLayer('Name', 'relu_fc1')
40
41
42          fullyConnectedLayer(512, 'Name', 'fc2')
43          reluLayer('Name', 'relu_fc2')
44          fullyConnectedLayer(5, 'Name', 'fc3')
45          softmaxLayer('Name', 'softmax')
46          classificationLayer('Name', 'output')];
47      options = trainingOptions('sgdm', ...
48          'InitialLearnRate', 0.001, ...
49          'MaxEpochs', 20, ...
50          'Shuffle', 'every-epoch', ...
51          'ValidationData', imdsValidation, ...
52          'Verbose', false, ...
53          'Plots', 'training-progress');
54      disp('Training the custom CNN...');
55      trainedNet = trainNetwork(imdsTrain, layers, options);
56      save('trainedCustomCNN.mat', 'trainedNet');
57      disp('Model training completed.');
58
```

OUTPUT:

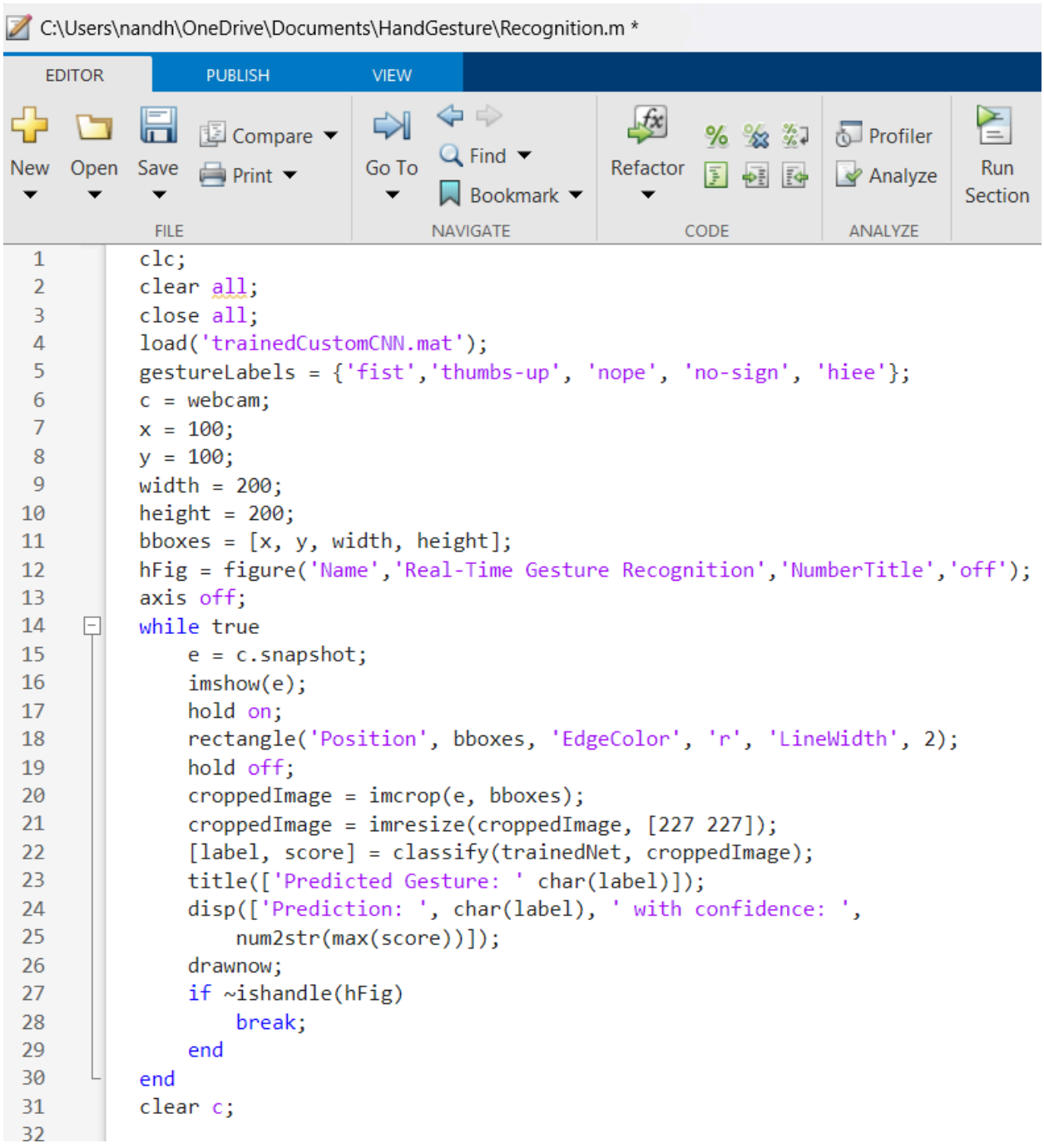




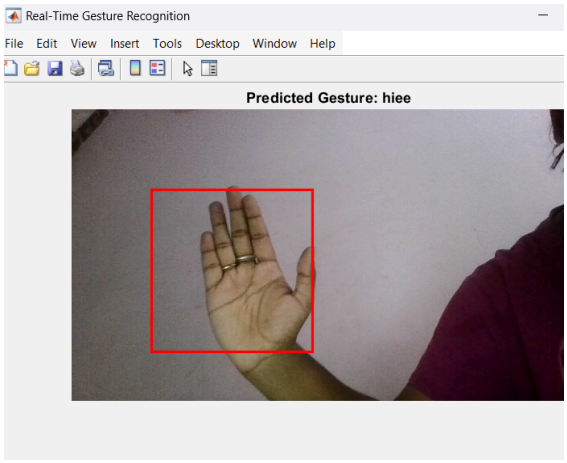
Matlab code for implementation(real-time hand gesture recognition):

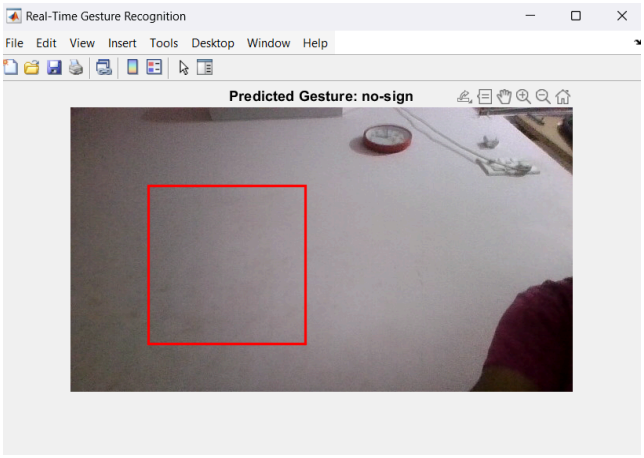
This MATLAB code is used for implementing real-time hand gesture recognition using a webcam and a pre-trained custom Convolutional Neural Network (CNN). The model classifies the gestures in real-time based on the region of interest (ROI) where the hand gesture appears. In this code we use

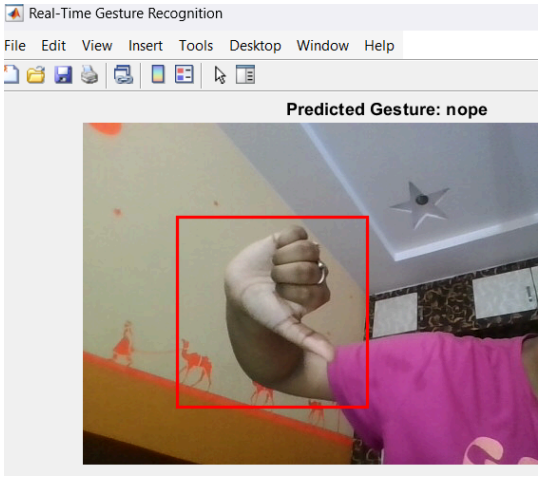
1. **Webcam**: The webcam continuously captures frames to detect and recognize gestures.
2. **Region of Interest (ROI)**: A fixed bounding box is used to define the area where hand gestures will be detected.
3. **Gesture Classification**: The cropped region of interest is passed through the trained CNN, which classifies the gesture.
4. **Real-Time Display**: The predicted gesture is displayed on the figure window in real-time.

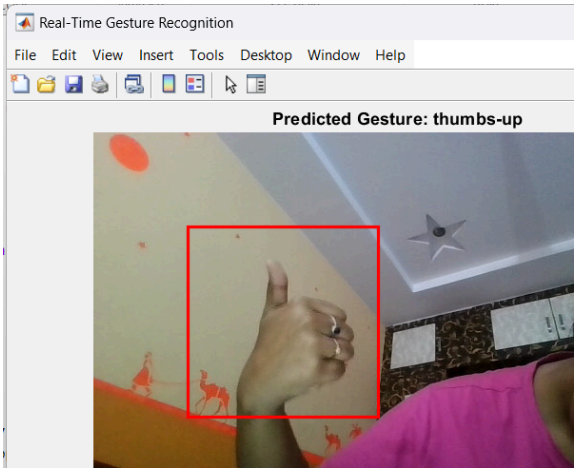

C:\Users\nandh\OneDrive\Documents\HandGesture\Recognition.m *
EDITOR
PUBLISH
VIEW
New
Open
Save
Compare
Print
FILE
Go To
Find
Bookmark
NAVIGATE
Refactor
CODE
Profiler
Analyze
ANALYZE
Run Section
clc;
clear all;
close all;
load('trainedCustomCNN.mat');
gestureLabels = {'fist','thumbs-up', 'nope', 'no-sign', 'hiee'};
c = webcam;
x = 100;
y = 100;
width = 200;
height = 200;
bboxes = [x, y, width, height];
hFig = figure('Name','Real-Time Gesture Recognition','NumberTitle','off');
axis off;
while true
e = c.snapshot;
imshow(e);
hold on;
rectangle('Position', bboxes, 'EdgeColor', 'r', 'LineWidth', 2);
hold off;
croppedImage = imcrop(e, bboxes);
croppedImage = imresize(croppedImage, [227 227]);
[label, score] = classify(trainedNet, croppedImage);
title(['Predicted Gesture: ' char(label)]);
disp(['Prediction: ', char(label), ' with confidence: ',
num2str(max(score))]);
drawnow;
if ~ishandle(hFig)
break;
end
end
clear c;

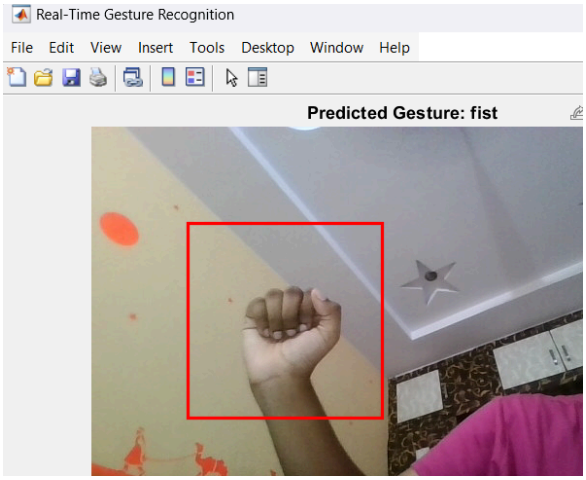
OUTPUTs:


Real-Time Gesture Recognition
File Edit View Insert Tools Desktop Window Help
Predicted Gesture: hiee


Real-Time Gesture Recognition
File Edit View Insert Tools Desktop Window Help
Predicted Gesture: no-sign


Real-Time Gesture Recognition
File Edit View Insert Tools Desktop Window Help
Predicted Gesture: nope


Real-Time Gesture Recognition
File Edit View Insert Tools Desktop Window Help
Predicted Gesture: thumbs-up


Real-Time Gesture Recognition
File Edit View Insert Tools Desktop Window Help
Predicted Gesture: fist

USES Of Hand-Gesture Recognition In Real-Life:

1. Sign Language Translation

- **Communication for the Deaf and Hard of Hearing**: Hand gesture recognition systems can convert sign language gestures into text and helps in communication between deaf individuals and people who don't know sign language.
- **Learning**: Helps in teaching sign language by giving real-time text of gestures.

---

2. Gaming and Entertainment

- **Gesture-Based Gaming**: Used in gaming consoles to translate hand movements into actions in the game.
- **Virtual Reality (VR)**: Hand gestures can be used to interact in virtual game than to use controllers.

---

3. Smart Home Automation

- **Smart Home Control**: Gesture-based control allows users to manage lights, thermostats, appliances, and other IoT devices without physical contact or voice commands.

---

4. Robotics

- **Human-Robot Interaction (HRI)**: Hand gestures are used to control robots or robotic arms in manufacturing, warehousing, or even service industries. This makes robots more adaptable and easier to operate without needing a complex interface.

---

5. Security and Authentication

- **Gesture-Based Authentication**: Gestures can be used as a form of authentication or access control, offering an additional layer of security through biometric recognition (e.g., signing in with specific gestures).

---

6. Education and Learning

- **Interactive Learning Tools**: Gesture recognition can be used in educational applications where students interact with digital content (such as textbooks, videos, or 3D models) via gestures. This can be particularly useful in interactive science labs, art, or history lessons.

CONCLUSION

Hand gesture recognition is a technology that bridges the gap between humans and machines, allowing for more intuitive, efficient, and seamless interfaces. This code helps us understand how to combine computer vision, machine/deep learning and hardware integration to create innovative systems and applications.